

MAJOR PROJECT REPORT

PyMeet: Advanced Real-time Video Conferencing Platform

Developed By: Ansh

A Comprehensive Study and Implementation of WebRTC & Modern Web Architecture

1. Abstract & Introduction

In the post-pandemic digital era, real-time virtual communication has become the backbone of enterprise and personal collaboration. PyMeet is a modern, highly scalable, and secure video conferencing application built from the ground up to solve the challenges of remote communication. Inspired by enterprise solutions like Zoom and Google Meet, PyMeet integrates advanced Peer-to-Peer (P2P) signaling, real-time media streaming, and secure data handling into a unified platform.

The primary objective of this project was to design a robust architecture capable of handling concurrent users with minimal latency, while providing an intuitive, aesthetically pleasing user interface characterized by modern design paradigms (e.g., glassmorphism, dark mode compatibility).

2. System Architecture

The system utilizes a decoupled Client-Server architecture with state-of-the-art technologies at every layer. The architecture ensures separation of concerns, scalability, and high maintainability.

Flow Overview: The client (React App) authenticates via REST APIs with the Backend (FastAPI). Once authenticated, the user connects to the signaling server via WebSockets. When a meeting is initiated, WebRTC protocols establish a direct P2P connection between clients for media transmission, ensuring maximum privacy and minimal server load.

2.1 Server Details

Component	Details / Live Environment
Database Server	Hosted remotely on a dedicated cloud instance (IP: 51.89.97.76). Engine: PostgreSQL 14+.
Backend API Server	Python ASGI Server (Uvicorn) handling HTTP/1.1 and WebSockets.
Frontend Host	Vite Development Server with Hot Module Replacement (HMR).

3. Comprehensive Technology Stack

The selection of tools was driven by the need for performance, security, and developer ergonomics.

3.1 Frontend Ecosystem

- **React.js & TypeScript:** Provides a strongly-typed, component-based UI architecture.
- **Tailwind CSS & Lucide Icons:** Enables rapid, utility-first styling with premium vector icons.
- **Vite & PWA Plugins:** Ensures lightning-fast builds and enables the application to be installed locally as a Progressive Web App (PWA).
- **ZegoCloud UIKits:** Provides robust pre-built communication layers for WebRTC media streams, handling edge cases like network drops and device management seamlessly.

3.2 Backend & Database Ecosystem

- **FastAPI (Python):** An ultra-fast web framework leveraging Python's `asyncio` for high-throughput API endpoints.
- **PostgreSQL:** The primary relational database, chosen for its ACID compliance and reliability in production environments.
- **SQLAlchemy & Alembic:** Advanced ORM and migration tools for structured database schema management.
- **Passlib & bcrypt:** Industry-standard cryptographic hashing for secure password storage.
- **PyJWT:** For generating and validating secure, stateless JSON Web Tokens (JWT) used in authentication.

4. Implementation Details & User Interface

4.1 Secure Registration & Authentication

Security is paramount. The registration module collects user data, hashes passwords using bcrypt, and stores the user securely in the remote PostgreSQL server. A JWT is issued upon login to maintain session state.

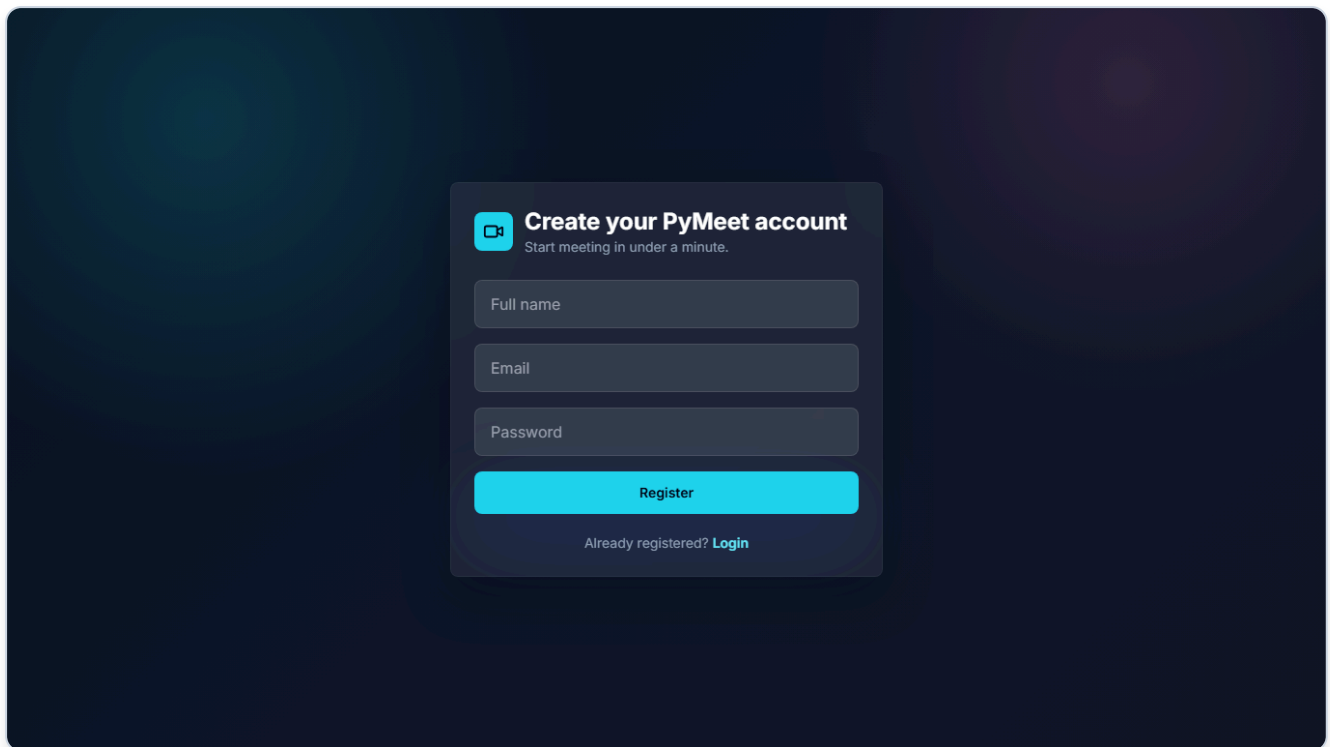


Figure 1: The Registration and Authentication Interface

4.2 Centralized Dashboard

The dashboard acts as the command center. It implements complex state management to fetch user profiles, display action buttons (Create Meeting, Join Meeting), and provides settings management including dark mode toggles.

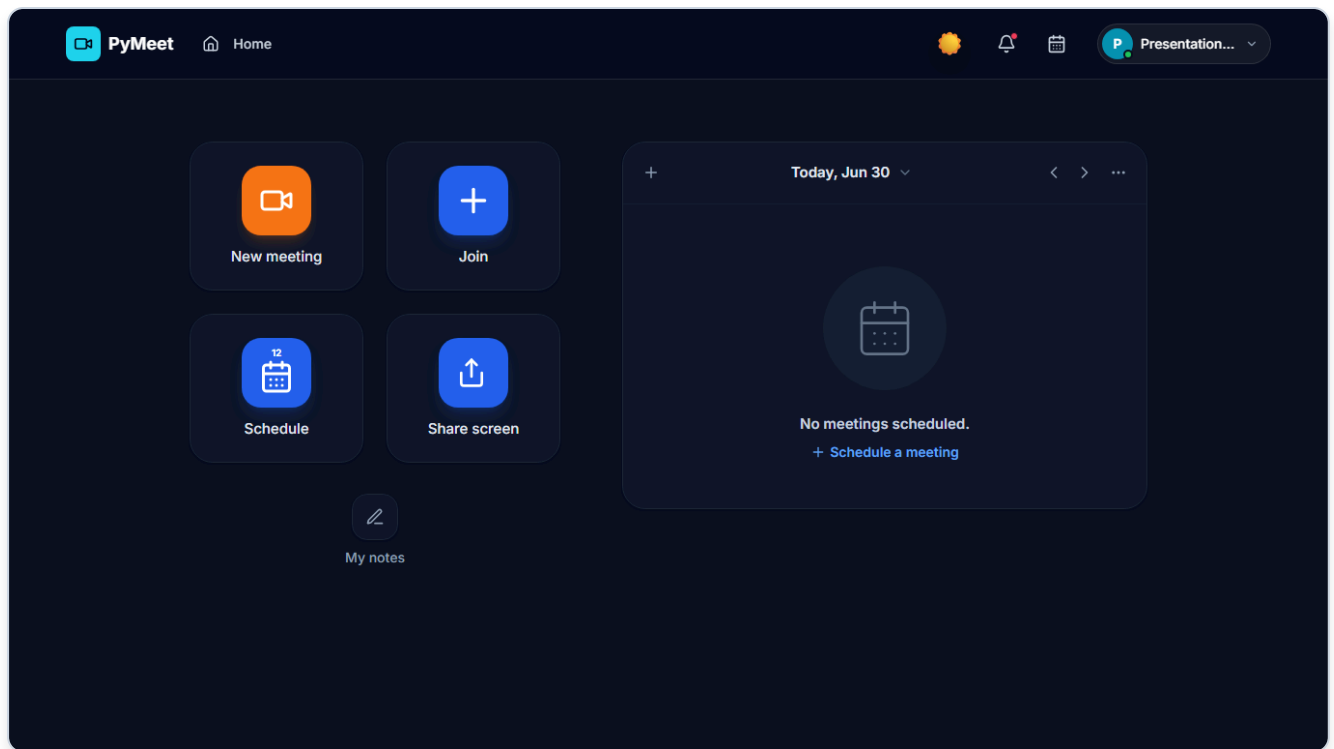


Figure 2: The Main User Dashboard Interface

4.3 Active Meeting Room & Media Streaming

The core feature of PyMeet. Utilizing WebRTC, this interface handles dynamic audio/video tracks, screen sharing, and real-time chat. The UI automatically adjusts layouts based on the number of participants.

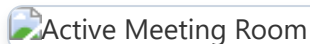


Figure 3: Active Video Conferencing Environment

5. Key Challenges Overcome

- **Cross-Origin Resource Sharing (CORS):** Configured robust CORS policies on FastAPI and proxying via Vite to allow seamless local testing and secure remote requests.
- **State Synchronization:** Managed complex React states for media devices (camera/mic toggles) to ensure UI correctly reflects hardware status.
- **Database Connectivity:** Transitioned from local SQLite to remote PostgreSQL without altering business logic, proving the robustness of the SQLAlchemy ORM layer.

6. Conclusion & Future Scope

The PyMeet project successfully achieves its goal of delivering a high-quality video conferencing tool. By utilizing a modern stack comprising React, FastAPI, and PostgreSQL, the platform is not only functional but highly scalable.

Future Enhancements:

- AI-based live transcription and translation.
- End-to-End Encryption (E2EE) for media streams.
- Virtual backgrounds utilizing WebGL and machine learning models.